

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Un server MCP per Claude Code

MCP in una riga

Uno standard aperto per dare a Claude Code strumenti e dati

- Nel Modulo 4, dopo l'installazione: dare a Claude Code l'accesso a **dati e sistemi** che da solo non vede
- Il **Model Context Protocol**, MCP, è uno **standard open source** per collegare l'AI agli strumenti: con MCP Claude Code si collega a centinaia di strumenti e fonti di dati esterni
- Un **server MCP** dà a Claude Code accesso ai vostri strumenti, ai vostri **database**, alle vostre **API**
- Anthropic lo ha donato all'**Agentic AI Foundation**, un fondo della Linux Foundation
- Qui MCP è una **configurazione di Claude Code**: un server locale, quello del corso, che conosce la copertura del gestionale di CiaspoLink. Nel Modulo 7 tornerà, come gli strumenti di un assistente

Un server locale: comando, argomenti, ambiente

Claude Code avvia il server: lo si dichiara con claude mcp add

Il server del corso è un programma Python che gira sul **vostro computer**, come processo locale: Claude Code lo **avvia** e gli parla. Si dichiara così (README di M4-D02, passo 3):

```
claude mcp add copertura -e COPERTURA_DATI="$D" \
  -- uv run --frozen --directory "$S" server_copertura.py
```

- Prima il **nome**, `copertura`; poi le opzioni di Claude Code, qui `-e`: una **variabile d'ambiente** per il server. Il server del corso non ha chiavi: `COPERTURA_DATI` porta il **percorso dei dati** del gestionale
- Il doppio trattino `--` separa le opzioni di Claude Code dal **comando** che avvia il server, uv, e dai suoi **argomenti**. `$S` e `$D`: i percorsi della cartella del server e di quella dei dati, scritti al passo 2
- L'**ordine** conta: se il nome venisse subito dopo `--env`, la CLI lo leggerebbe come un'altra coppia `CHIAVE=valore`, e lo rifiuterebbe
- Percorsi **assoluti**: uno relativo, nel comando o negli argomenti, si risolve dalla cartella da cui avete lanciato Claude Code, ed è una causa frequente di server che non partono. Al server Claude Code passa anche `CLAUDE_PROJECT_DIR`, la radice del progetto
- `--frozen` è di uv, non di Claude Code: «Run without updating the `uv.lock` file». uv usa le versioni fissate dal corso, e non riscrive il file

Dove si salva, e chi lo approva

Tre scope; e un server di progetto chiede la vostra approvazione

- `claude mcp add` salva il server in uno di tre **scope**, che si sceglie con `--scope`. **local**, il predefinito: vale solo nel progetto in cui lo aggiungete, e solo per voi; sta in `~/.claude.json`
- **project**: in `.mcp.json`, nella radice del progetto, da mettere in git per chi lavora con voi. E c'è anche **user**
- In JSON, un server stdio è un `type`, un `command` e i suoi `args`. Qui il file della demo che segue, che il docente passa a `claude -p` con `--mcp-config`:

con.json della demo, i percorsi abbreviati

```
{
  "mcpServers": {
    "copertura": {
      "type": "stdio",
      "command": "uv",
      "args": ["run", "--frozen", "--directory",
               "<cartella del server>", "server_copertura.py"],
      "env": {"COPERTURA_DATI": "<cartella dei dati>"}
    }
  }
}
```

Dove si salva, e chi lo approva

Tre scope; e un server di progetto chiede la vostra approvazione

- Un server di **progetto** arriva con il repository: in una sessione interattiva Claude Code **chiede** prima di usarlo, e finché non lo approvate `claude mcp list` lo dà in attesa, «Pending approval», e non lo collega
- Con `claude -p` quella domanda non si può fare: i server di progetto si caricano **senza chiedere**, anche in una cartella che non avete mai autorizzato
- Con `--strict-mcp-config` Claude Code usa **solo** i server passati con `--mcp-config`, e ignora ogni altra configurazione MCP. Così la demo che segue sceglie, a ogni esecuzione, con il server o senza
- Nella sessione, `/mcp` mostra e gestisce le connessioni ai server; dal terminale, `claude mcp list`, `get` e `remove`

M4-D02 Un server MCP per la copertura

Demo del docente in Claude Code, poi la vostra prova: con e senza il server

⚖ Due bracci che differiscono in **una cosa sola**, il server: `--mcp-config` su `con.json`, che lo dichiara, o su `senza.json`, che non ne ha; in tutti e due `--strict-mcp-config`. Lo spazio di lavoro **non ha dati**: la risposta può venire soltanto dal server

domanda.txt, uguale nei due bracci, a capo per la slide

```
<instructions>
Quali comuni hanno indirizzi coperti dalla fibra FTTH di
CiaspoLink?
</instructions>

<format>
Rispondi soltanto con l'elenco dei comuni, uno per riga, in
ordine alfabetico.
</format>
```

>_ `claude -p`, tre coppie: Sonnet, effort low, il solo strumento integrato Read (`--tools Read`), consentito quello del server. Guardate se Claude lo **chiama**, e quali comuni nomina

💡 Soltanto Read, e le uscite di ogni braccio **fuori** dallo spazio di lavoro fino alla fine: così un braccio non può leggere ciò che ha salvato l'altro

M4-D02 Un server MCP per la copertura

Demo del docente in Claude Code, poi la vostra prova: con e senza il server

- **Indovinate, a voce**: senza il server, Claude saprà quali comuni della valle sono coperti in fibra?
- **Intanto voi**, mentre la demo gira, dalla cartella `esercitazioni`: `tools/prepara_esercizio.sh M4-D02`, poi il passo 1 del README, `uv sync --frozen`: una volta sola, con la rete
- **Nella registrazione del corso: no**. Senza il server, «0 comuni nominati» in tutte e tre le esecuzioni: «Non riesco a rispondere […]». Con il server, una chiamata a `mcp__copertura__cerca_copertura`, e «giusti 13 su 13» in tutte e tre

M4-D02 Un server MCP per la copertura

Demo del docente in Claude Code, poi la vostra prova: con e senza il server

> Dalla pagina sui server MCP di Claude Code, la frase che la demo mette alla prova: «MCP servers give Claude Code access to your tools, databases, and APIs.» E due **garanzie**, da verificare a ogni esecuzione: «Start the session with --strict-mcp-config. Claude Code then uses only the MCP servers you pass with --mcp-config.»; e, dal riferimento degli hook, «MCP tools follow the naming pattern mcp__<server>__<tool>»

La registrazione del corso, 1 novembre 2026, Claude Code 2.1.316, Sonnet: le righe del controllo, che legge lo stream di ogni esecuzione

le righe del controllo: «[…]» segna un taglio

```
[…] «Con --strict-mcp-config, solo i server di --mcp-config»
  (con, 1): confermata
«Gli strumenti MCP si chiamano mcp__<server>__<tool>»
  (con, 1): confermata
[…]
Dati: lo strumento del server chiamato: 3 su 3
Dati: risposte con esattamente i comuni giusti: con il server
  3 su 3, senza 0 su 3
[…]
```

La prima garanzia, **«confermata»** in tutte e sei le esecuzioni, con il server e senza; la seconda, nelle tre con il server: nove righe su nove. Lo strumento chiamato e i comuni giusti sono **dati**

M4-D02 Un server MCP per la copertura

Demo del docente in Claude Code, poi la vostra prova: con e senza il server

1. Lo spazio di lavoro e il passo 1 li avete lanciati durante la demo. Dalla cartella `esercitazioni`, i percorsi S e D (passo 2), poi `cd ~/corso-claude-code/M4-D02`
2. Il comando della pagina «Un server locale»: `claude mcp add copertura -e …`; poi, per controllare, `claude mcp get copertura`
3. `claude --model sonnet --effort low`; `/mcp`, e guardate `copertura`; la domanda di `domanda.txt`; poi «Scrivi quell'elenco, un comune per riga, nel file risposta.txt.»; `/exit`
4. `cd -` e `tools/verifica.sh M4-D02`; alla fine, dallo spazio di lavoro, `claude mcp remove copertura`

- La verifica chiede a `claude mcp get copertura` se il server è configurato in questo spazio di lavoro, e controlla che `risposta.txt` nomini **esattamente** i comuni giusti
- Chi non finisce in aula **continua da solo**, con il README: i passi sono gli stessi
- Nel repository delle esercitazioni: `m4_claude_code_agenti/s3_agente_nel_repository/m4_4_server_mcp/M4-D02_server_copertura/`

I server degli altri

Prima di collegarne uno, chiedetevi se vi fidate

- Prima di collegare un server, verificate di **fidarvi**: un server che recupera **contenuti esterni** può esporvi alla **prompt injection**
- Anthropic incoraggia a **scrivere i vostri** server, o a usare quelli di fornitori di cui vi fidate. Controlla i connettori della sua Directory secondo i suoi criteri, ma **non fa audit di sicurezza** su nessun server MCP, e non ne gestisce nessuno
- Se l'output di uno strumento MCP supera i **10.000 token**, Claude Code vi avvisa; il massimo, di serie, è di 25.000
- I connettori che avete in claude.ai arrivano anche in Claude Code, ma solo quando l'accesso attivo è quello dell'**abbonamento**. Per una sessione con i soli server che scegliete voi: `--strict-mcp-config` e `--mcp-config`, come nella demo

Costruirne uno

Il server del corso: poche righe di FastMCP, in un progetto uv

- Per costruirne uno, la documentazione rimanda alla **guida** dei server MCP, per i fondamenti del protocollo; o lo scheletro lo imposta Claude, con il plugin ufficiale `mcp-server-dev`
- Nella skill di Anthropic per costruire server MCP, l'SDK ufficiale **TypeScript** è la scelta predefinita; per chi preferisce Python, **FastMCP 3.x**. Uno script locale, lanciato sul vostro computer, va bene per strumenti **personali** e prototipi
- Il server del corso: FastMCP in un progetto **uv**, con le versioni fissate in `uv.lock`; uno strumento, in **sola lettura**, con due filtri facoltativi:

`server/server_copertura.py, uno stralcio`

```python
server = FastMCP("copertura", instructions=ISTRUZIONI)
# […]
@server.tool(annotations={"readOnlyHint": True,  # […]
                          "openWorldHint": False})
def cerca_copertura(
        comune: Optional[str] = None,
        tecnologia: Optional[Literal["FTTH", "FWA"]] = None,
) -> list[dict]:  # […]
```

Un server MCP locale è un programma che Claude Code avvia: i suoi strumenti diventano di Claude, e i dati restano dove sono.